

PetPluse – سكربت العرض

Phase 2 Defence · ٢٨ شريحة · مقسم على أعضاء الفريق

إشراف: د. محمد صبح · م. هند عادل

2026 / 2025 · petpluse.com

طريقة الاستخدام

- النص المكتوب داخل الإطار هو ما يُقال فعليًا — مكتوب ليُنطق، مش ليُقرأ حرفيًا.
- المصطلحات التقنية مكتوبة بالإنجليزية عن قصد، لأنها هي المتعارف عليه.
- «نقاط أساسية» هي اللي لازم تُذكر حق لو الوقت ضاق.
- الأرقام كلها حقيقية ومأخوذة من الكود — تقدر تسندها لو اتسألت.
- إجمالي الوقت التقريبي: 18-22 min بدون الأسئلة.

المقدمة والمشكلة

المتحدثة: سلمى محمد — Frontend · الشرائح ١-٥ · ~4 min

~30s

Title — PetPluse 01

صباح الخير. إحنا فريق PetPluse، ومشروعنا منصّة رعاية حيوانات أليفة موحّدة للسوق المصري. في الـ Phase 1 كان عندنا فكرة أثبتنا إنها ممكن تشتغل. النهاردة هنعرض حاجة مختلفة: نظام حقيقي، شغال على الإنترنت دلوقي، وאתعرض لاختبار اختراق فعلي وصمد. العنوان اللي تحت اسم المشروع بيلخص الرحلة: **from prototype to a system that survives an attack**.

منصّة موحّدة · شغالة فعليًا · اتعرضت لاختبار اختراق

~25s

Agenda 02

هنمشي في تمن أقسام: إيه اللي اتغيّر من Phase 1، الفريق، المشكلة والحل، الـ architecture، اللي سلّمناه فعليًا، عرض حيّ للنظام، الـ security، وأخيرًا النتائج. وأهم جملة في العرض كله موجودة تحت: Phase 1 كانت بتقول إن الفكرة ممكنة. Phase 2 لازم تعرض نظام.

٨ أقسام · الادعاء: نظام مش فكرة

~50s

Phase 1 → Phase 2 03

الرمادي هو Phase 1، والأزرق هو النهاردة. الـ endpoints من ٥٠ لـ ٢١٢. جداول قاعدة البيانات من ١٢ لـ ٥٧. الـ user roles من ٥ لـ ٦. وبصّوا على الثلاثة اللي تحت — دول كانوا صفر خالص: عشر AI tools، عشر test suites، و٣٢٨ assertions بتعدّي كلها. يعني اللي اتضاف مش مجرد features زيادة — اتضافت طبقة ذكاء اصطناعي وطبقة اختبارات ما كانوش موجودين أصلًا.

endpoints 212 → 50 · tables 57 → 12 · assertions 328 → 0

لو الوقت ضاق: اذكري الـ endpoints والـ assertions بس.

إحنا عشرة، متقسمين على ثلاث تخصصات مختلفة بس على codebase واحد.
 ال Engineering مسؤولين عن المنصة وال product. ال Intelligence بنوا طبقة ال Agentic AI. وال Assurance فريق ال Security – ودول هما اللي هاجموا النظام بتاعنا فعليًا، وهيعرضوا نتايجهم بنفسهم في القسم السابع.

codebase واحد

٣ تخصصات

١٠ أعضاء

المشكلة – أربع مشاكل حقيقية 05

إحنا ما بدأناش من فكرة – بدأنا من أربع مشاكل موجودة فعليًا في السوق المصري.
الأولى: الحجز عند الطبيب بيتم بالتليفون، ومفيش تأكيد بيوصل. عندنا دلوقي مواعيد حقيقية متاحة، حجز فوري، وإيميل بملف تقويم.
الثانية: الإعلانات مش موثقة. عندنا تحقّق بمستندات – محدش بيظهر على المنصة قبل الموافقة.
الثالثة، وهي الأهم: حالات الطوارئ بتأخذ ردّ عام. عندنا كشف deterministic بيشتغل قبل أي model، بالعربي والإنجليزي.
الرابعة: المحلات ملهاش طريق للأونلاين. عندنا storefronts، ورفع كتالوج كامل من ملف واحد.
 والنقطة المهمة تحت: كل حل من دول **شغال النهاردة** – مش خطة.

مفيش أونلاين → storefront

ردّ عام → كشف طوارئ

غير موثّق → مستندات

تليفون → حجز فوري

انتقال: «سارة هتوريكم السوق نفسه وإحنا فين منه.»

السوق والمنتج

المتحدثون: سارة صلاح (٦-٧) · ليلى محمد (٨) · فاطمة محمد (٩-١١) · 5 min ~

Market Research & Persona

06

~55s

السوق المصري مجزأ. دي مش نظرية — دي نتيجة بحث على المنافسين الموجودين فعلاً. persona بتاعتنا سارة، ٢٣ سنة، graphic designer من القاهرة. مشاكلها الثلاثة على الشمال هي بالضبط اللي المنصة بتحلّها. وعلى اليمين المنافسين: 7Pets وPetSmart تجربتهم الأونلاين ضعيفة. EGY Puppy أسعارهم عالية وتركيزهم محدود. والعالميين زي Rover خدماتهم مفكّكة. مفيش حد فيهم بيقدم منصة موحّدة فيها AI — وده اللي إحنا بنيناه.

الفجوة: منصة موحّدة + AI

٤ منافسين

persona حقيقية

سوق مجزأ

~55s

Six Roles

07

المنصة واحدة، بس بتخدم ست أدوار مختلفة — وكل دور بياخد حاجة مختلفة تمامًا. صاحب الحيوان بيحجز ويشترى ويتابع التطعيمات من حساب واحد، بالعربي أو الإنجليزي. الطبيب بياخد profile موثّق وتقويم حقيقي ورصيد عيادة مدفوع مقدّمًا. المدرّب بيعمل كورسات أسابيع بعدد مقاعد حقيقي. المحل بياخد storefront بلينك خاص بيه. والدور الجديد في Phase 2 هو الـ Clinic Assistant — موظّف الاستقبال بياخد login خاص بيه، ومش بيشوف السجّلات الطبية إطلاقًا.

استقبال بدون سجّلات طبية

Clinic Assistant جديد

٦ أدوار

انتقال: «ليلى هتوضّح الـ architecture اللي شايك ده كله.»

~60s

النظام ستّ طبقات، بس deployment واحد.

من فوق: ال Client — React 19، ٣٦ صفحة، عربي وإنجليزي بال RTL. تحتها ال Edge على Vercel. بعدها ال API — module ٢٠ و endpoint ٢١٢، وكلها ورا JWT و RBAC و validation.

الطبقة الرابعة هي ال VetAI — Intelligence. تحتها ال Data على PostgreSQL. وفي الأساس الخدمات الخارجية.

والسطر اللي تحت مهم: ده مسار أي request جوّه النظام — من ال HTTPS لحد ال parameterised SQL. مفيش query واحدة بتتبني بلصق نصوص.

parameterised SQL دائماً

endpoints 212

٦ طبقات، deployment واحد

~45s

ولا حاجة في ال stack ده بترخيص مدفوع.

React 19 و Vite و Tailwind قدام. Node و Express ورا. PostgreSQL على Supabase. ال auth بال JWT و bcrypt ومعاه Google Sign-In.

ال AI على Groq و Ollama. الصور على Cloudinary، الخرائط Leaflet و OpenStreetMap، الدفع Paymob، والإيميل Brevo.

وأهم واحدة: ال OCR بتاع التحقق من الرخص بيشتغل بـ Tesseract.js على السيرفر بتاعنا — مفيش مستند بيخرج برّه.

مفيش مستند بيخرج

OCR محلي

صفر ترخيص مدفوع

~50s

ستّ أنظمة فرعية، كلها شغالة على اللينك الحقيقي — مش ورا demo mode.

ال Appointments بتعرض المواعيد الفاضية الحقيقية، ومستحيل اتنين ياخدوا نفس الميعاد. ال Marketplace محلات موثقة بس. ال Community فيها حق اعتراض على أي حذف بيعمله ال AI.

وال Lost & Found — دي مش لوحة إعلانات. أول ما حد يبألغ عن حيوان لقاه، النظام بيقرانه بكل البلاغات المفتوحة في النطاق ويبألغ صاحبه هو بنفسه.

Lost & Found بيدور لوحده

مستحيل حجز مزدوج

٦ أنظمة شغالة

التبنيّ عندنا التاريخ الطبي بتاع الحيوان متعلّق بال `profile`، فاللي هيتبنيّ يشوف التطعيمات بالضبط. التزاوج بيشتغل بطلب لازم صاحب الحيوان الثاني يوافق عليه قبل ما يتشارك أي حاجة – والتحقّق من إن النوع واحد متفروض في ال `API` نفسه، مش في ال `form` بس. والمسافات بتتْحَسَب عندنا بـ `Haversine` جوّه ال `SQL` – يعني مغيّش خدمة خرايط خارجية بنسألها الناس فين، والإحداثيات مش بتخرج من قاعدة البيانات بتاعتنا.

[Haversine جوّه SQL](#)[موافقة الطرفين](#)[تاريخ طبي شفاف](#)

انتقال: «ليلي هتتكلم عن الجزء اللي أخذ أكبر شغل – ال `Agentic AI`».

Agentic AI II

المتحدثون: ليلي محمد (١٢، ١٤، ١٥) · أحمد شريف (١٣) · 5 min ~

~50s

AI Evolution 12

في الأول كنّا بنستخدم API OpenAI مدفوع. ده كان wrapper بسيط — بنبعت السؤال وبنستقّي الرد، من غير أي تحكّم في اللي بيحصل جوّه.

تكلفة شهرية ثابتة، و vendor lock-in كامل.

دلوقتي إحنا بنينا ال agentic loop بنفسنا. تحكّم كامل في ال architecture وال tool schemas، صفر تكلفة متكرّرة عن طريق open-weight models، والأهم إن المزود قابل للتبديل — Groq أو Ollama أو mock بمتغيّر واحد.

مفيش lock-in

صفر تكلفة

ال agentic loop داخلي

من wrapper مدفوع

~60s

Agentic AI ① — محادثة واحدة 13

الفرق بين chatbot و agent بسيط: ال chatbot بيوصف لك الحجز، وال agent بيعمله.

بصّوا على المحادثة على الشمال. أول سؤال: «كلي أكل شوكولاتة». الرد فوري — دي حالة طوارئ، اتّصل بطبيب دلوقتي. والشريط الأخضر تحتيه بيقول إن ده safety guardrail اشتغل قبل ال model أصلاً.

تاني رسالة: «احجزلي دكتور بكرة الصبح». ال agent استخدم tools حقيقية، لقي ميعاد فاضي، وحجزه فعلاً وبعث دعوة تقويم.

على اليمين العشر tools اللي عنده — وكل argument فيهم متحقّق منه ب schema.

tools ١٠

الأمان قبل ال model

حساب + حيوان + طبيب + حجز

الناس بتفكر إن ال AI assistant بيشتغل كده: الرسالة تروح لل model، وال model بيقرر كل حاجة. عندنا مش كده — وده عن قصد.

الرسالة بتعدّي على تمن مراحل. سنّة منهم عمرهم ما بينادوا model خالص. واحدة بتستخدمه علشان تقرا حقل واحد من الجملة. وواحدة بس — البرتقالية — هي ال AI الفعلي.

الأول بنحدّد اللغة بنفسنا من الحروف. بعدها ال safety check — ده pattern match بيشتغل قبل ال model. لو اشتغل، نبردّ على طول وال model ما بيتناداش أصلاً.

وده السبب — بضوا على العمودين تحت: المسار ال deterministic تلتمية وتلاتين millisecond وصفر tokens. مسار ال model من تمن لثمنناش second. أربعين ضعف. علشان كده معظم الرسايل بتتجاوب قبل ما ال AI يشتغل أصلاً.

330ms مقابل 8-18s

الأمان أولاً وبشكل حتمي

٨ مراحل، ال model في واحدة

لو اتسألتي «ليه مش ال AI يتولّى الأمان؟»: لأنه الحاجة الوحيدة اللي ممنوع تفشل. ال pattern match بيشتغل دايماً وبنفس السرعة. ال model ممكن ببطاً أو يتقنع بالعكس.

دي المشكلة اللي مش بتبان غير لما النظام يشتغل حقيقي.

إحنا شغّالين serverless. يعني ممكن يبقى في ستّ نسخ من السيرفر شغّالة في نفس اللحظة، وكل واحدة فيهم معندهاش أي فكرة عن الثانية. وفي نفس الوقت ميزانية ال tokens بتاعتنا مشتركة على مستوى الحساب كله.

لو كل نسخة عدّت لوحدها، هيتصرف أكثر من المتاح.

الحل: ال queue نفسه قاعدة بيانات. كل النسخ بتتزامن على صف واحد في جدول اسمه ai_chat_gate. والجملة اللي على الشمال دي هي كل الحكاية — UPDATE بياخد lock على الصف ده، وما بينجش غير لو في مكان و لو دورك جه. وال INSERT ما بيحصلش أصلاً إلا لو ال UPDATE نجح.

يعني مستحيل نسختين يقرأوا «في مكان» في نفس اللحظة — لأنهم بيتزاحموا على قفل واحد.

والنتيجة تحت: صفر جنيه اتصرفوا على ال model inference. مفيش API مدفوع في النظام كله.

EGP 0

FIFO حقيقي

صف واحد بيتقفل

serverless مش شايف بعضه

دي أقوى نقطة تقنية في العرض — خدي وقتك فيها لو الوقت يسمح.

الإدارة والعرض الحيّ

المتحدثون: أحمد شريف (١٦) · عصام أسامة ومحمود إبراهيم (١٧-١٨) · 3 min ~

~55s

Administration – AdminPulse

16

النظام مش مبني علشان يتعرض بس – مبني علشان يتشغل.

على الشاشة دي ال `feature flags`. أي قسم في المنصة ممكن يتحوّل لـ «قريبًا» من غير أي `deployment` – بتفتح المفتاح لما أول شريك حقيقي يبقى جاهز.

التحقّق من الرخص بيتّم بـ OCR محلّي، والموافقة النهائية بشرية. وكل إجراء إداري بيتسجّل في `audit log` باسم اللي عمله.

وفي ال `moderation`، لما ال AI يحذف محتوى، صاحبه بيقدّر يعترض والاعتراض بيرجع لطابور بشري.

حق الاعتراض

audit log

OCR محلّي + موافقة بشرية

feature flags من غير deploy

~50s

Demo – Admin Console

17

دي مش صورة تصميم – دي الشاشة الحقيقية شغالة على قاعدة البيانات الحقيقية.

ال AI Copilot ده بيخلّيك تسأل عن المنصة بلغة عادية. السؤال اللي في الصورة كان «`total revenue breakdown`» – والرد رجع مقسّم: المدفوعات المكتملة، والاشتراكات، والإجمالي، وعدد الحجوزات.

كل رقم على الشاشة دي مقروء من قاعدة البيانات وقت السؤال، مش محفوظ في مكان تاني.

تصدير CSV

سؤال بلغة عادية

بيانات حقيقية

معظم اللي بيستخدم المنصة بيدور على دكتور من موبايله — وغالبًا وهو قلقان على حيوانه. الأربع شاشات دي من التطبيق الحقيقي: ال Marketplace، والتبّي والتزاج، وال Community بال Lost & Found، والمحلات القريبة مرتبة بالمسافة الحقيقية. وفي كل شاشة فيهم ال VetAI موجود ومتاح.

[VetAI في كل مكان](#)[٤ شاشات حقيقية](#)[mobile-first](#)

انتقال: «دلو قتي فريق ال Security هيعرض اللي عملوه — وهما هاجمونا فعلاً.»

الدفاع الأمني

المتحدثون: جابر حسني (١٩، ٢٣) · عبدالرحمن عصام (٢٠) · محمود مصطفى (٢١-٢٢) · 5 min ~

Security Workstream

19

~60s

ال security عندنا ما كانش أسبوع في الآخر — كان شغل مستمر ثلاث شهور.

ثلاث مراحل بتواريخ حقيقية: Harden — راجعنا كل authentication control وقفلنا الفجوات. Attack — هجوم فعلي على النظام المنشور: black-box من غير كود، وبعدين grey-box بالكود، وبعدين مرحلة منفصلة على AI نفسه. Close — كل ثغرة اتصلحت في الكود واثتعداد اختبارها على النظام الشغال.

النتيجة: تمن ثغرات مؤكدة، أعلى واحدة فيهم 9.8 CVSS.

وأهم جملة: ولا ثغرة واحدة منهم طلعت من scanner أوتوماتيكي. كلها اتلقت بالإيد.

مفيش scanner لقي حاجة

٨ ثغرات

٣ مراحل بتواريخ

١٢ أسبوع

~60s

Security Hardening

20

قبل ما نهاجم، راجعنا عشرين control خاص بال authentication. أربعناشر منهم كانوا ناقصين.

الجدول ده بيعرض الحالة قبل وبعد لكل واحد. مثلاً: كلمة السر كان شرطها الطول بس — بقي فيه شروط تعقيد و denylist لكلمات السر المسربة. ال hashing من 10 cost bcrypt ل 12.

وأخطر واحدة: ال OTP كان بيتولّد ب Math.random — ودي مش آمنة تشفيرياً. بقي crypto.randomInt. وكمان ما كانش فيه حدّ لعدد المحاولات على كود من ستّ أرقام.

والخمسة اللي تحت كانوا سليمين أصلاً وسبناهم زي ما هما — لأن الهدف كان نصلّح فجوات حقيقية، مش نعيد كتابة كود شغال.

٥ كانوا سليمين

Math.random → crypto

١٤ اتصلّحوا

٢٠ اتراجعوا

تمن ثغرات، وكل واحدة فيهم قابلة لإعادة الإنتاج — يعني معها الخطوات اللي تخليك توصلها بنفسك. أخطر واحدة هي F-10، وتقييمها CVSS 9.8: connection string فيه باسورد الـ super-user بتاع قاعدة البيانات، مكتوب في الكود ومرفوع على repository عام — مش بس في آخر نسخة، لأ في تاريخ الـ git كله. اللي المهاجم بياخده هنا: قراءة وتعديل وحذف كل بيانات المنصة. ومش محتاج exploit أصلاً — إنه ينسخ الـ repository ده هو الهجوم كله. ودي بالضبط اللي بتوضح النقطة: TruffleHog لقاه، بس GitLeaks وnuclei ما لقوش حاجة خالص.

F-10 = 9.8

CVSS + CWE + OWASP

٨ ثغرات قابلة للإنتاج

الشاشة دي تفاعلية — تقدر تدوس على أي ثغرة أو تستخدم أرقام ٨-١.

كل ثغرة اتقفلت في الكود، واتّعاد اختبارها على النظام الشّغال — مش اتشطبت من ورقة. في حالة F-10، تعديل الكود لوحده ما كانش كفاية: مسح ملف مش بيمسح السر. فالباسورد اتغيّر فعلياً، والتاريخ اتنصّف. وعلشان ما تحصلش تاني، ضفنا pre-commit hook بيرفض أي ملف فيه database URL بباسورد جّواه. وال hook ده اشتغل فعلاً أثناء المشروع ورفض commit حقيقي. الباقي واضح: F-09 من ٢٠٠ لـ ٤٠٣. الـ IDOR من ٢٠٠ لـ ٤٠٤. وال dependencies من ٣٢ تحذير لصفر.

hook منع تكرارها

تغيير + تنضيف تاريخ

اتصلّح واتّعاد اختباره

إن إحنا نلاقي الثغرات ده نص الشغل. النص التاني هو اللي بيحمي النظام دلوقتي. عندنا SQLi gate بي فحص كل request ضدّ أكثر من عشرين توقيع. وال Security Agent — وده مهم — advisory only: بيشرح الحدث بعد ما يتمنع، مش بيشارك في قرار المنع نفسه. والقاعدة اللي ماشيين عليها في النظام كله: الـ model ممكن يشرح المنع، بس عمره ما يقرّره. نفس القاعدة اللي بتحكم الـ VetAI safety layer.

dependency ratchet

pre-commit hook

الـ AI استشاري بس

SQLi gate حتمي

انتقال: «عصام هيقلل بالنتائج والأرقام.»

النتائج والقرارات والختام

المتحدثون: عصام أسامة (٢٤-٢٧) · ليلي محمد (٢٨) · ~4 min

~50s

Results 24

Phase 2 بالأرقام: ٢١٢ endpoint على ٢٥ module. ٥٧ جدول و٥٦ migration. ٣٦ صفحة و٦ dashboards. عشر AI tools. ٣٢٨ assertion بتعدّي وصفر بتفشل. و٦٠٠ commit. وتحت كل رقم فيهم مثال حقيقي منه — endpoint بالاسم، أسماء جداول حقيقية، أسماء tools حقيقية. علشان الرقم يبقى مسنود على الشاشة نفسها. ولازم نكون صرحاء: دي أرقام تسليم، مش أرقام استخدام. المنصة منشورة بس لسه قبل الإطلاق الفعلي.

تسليم مش استخدام

أمثلة حقيقية

600 / 328 / 10 / 36 / 57 / 212

~55s

Engineering Decisions 25

ست قرارات تقنية، وكل واحد منهم ليه سبب. ال pooling متطبّل لـ connection — serverless واحد لكل نسخة، فقاعدة البيانات ما بتخلصش مهما زاد الضغط. ال realtime بيتدهور بشكل نظيف: sockets لو فيه process مستمر، و polling غير كده — والمستخدم مش ييفرق. ال OCR جوه عندنا، والأمان حتمي، والأسرار مش بتوصل لـ repository أصلاً.

شغالة فعليًا دلوقتي

كل واحد بسبب

٦ قرارات

~40s

Timeline 26

اتناشر أسبوع، ست محطّات — وكل محطّة سايبة وراها حاجة ملموسة. أسبوع ١ و٢ ال hardening و٥٦ migration. من ٣ لـ ٥ ال Agentic AI والعشر tools. من ٦ لـ ٨ ال commerce وال AdminPulse. من ٩ لـ ١٠ الهجوم والتمن ثغرات. أسبوع ١١ الإصلاح والتمنية اتقفلوا. وأسبوع ١٢ النشر والدفاع ده.

مخزج ملموس لكل واحدة

٦ محطّات

١٢ أسبوع

الشاشة دي فيها كل حاجة قلنا إحنا هنصلّحها – وكلها اتقفلت. ستّة من ستّة.
 على الشمال الوضع قبل، وعلى اليمين الوضع دلوقتي، وجنب كل واحد الدليل: الروابط القديمة لسه شغالة، ١١ من ١١
 اختبار concurrency، السحب على الرصيد مستحيل، التحذيرات من ١٩ ل ٢، و ٤٢ assertion بتحرس الـ
 .AI tools
 والجملة اللي تحت هي خلاصة المشروع: نظام ممكن يعدّي كل الفحوصات الأتوماتيكية وبرضه يكون مكشوف. اللي
 لقيناه، لقيناه بالإيد، وقفلناه قبل ما حد برّه الفريق يشوفه.

دليل مع كل واحدة

٦ من ٦

شكرًا لحضراتكم.
 المنصة شغالة على petpluse.com، والكود كله على GitHub، وفيه فيديو demo – وال QR code ده بيوصل
 عليها.
 إحنا جاهزين لأسئلتكم.

QR

demo video

GitHub

petpluse.com

أسئلة متوقعة وإجابات مختصرة

لكل الفريق — اقرأوها قبل الدفاع

ليه ما استخدمتموش OpenAI أو Gemini؟

استخدمناها في الأول فعلاً، وسبناها عن قصد. ال open-weight models على Groq كافية للشغل اللي إحنا عايزينه، وبتدّينا حاجتين: صفر تكلفة متكرّرة، وإمكانية تبديل المزود بمتغيّر واحد. النظام كمان بيشتغل بالكامل ب mock provider في CI من غير أي مفتاح.

إزاي متأكّدين إن ال AI مش هيقول حاجة غلط طبيّاً؟

إحنا ما بنعتمدش عليه في ده أصلاً. كشف الطوارئ والأدوية السامة pattern matching بيشتغل قبل ال model. وفيه طبقة تانية بتفحص رد ال model نفسه قبل ما يتبعث — لو ذكر جرعة أو وصفة منزلية خطيرة، الرد بيتمنع. وال RAG مسموح له يقول «مش عارف» بدل ما يخترع إجابة.

هل الأرقام دي حقيقية ولا تقديرية؟

كلها مقروءة من الكود وقاعدة البيانات. ال ٣٢٨ assertion ممكن تتشغل قدامكم دلوقتي. ال ٢١٢ endpoint و ٥٧ جدول اتعدّوا من المصدر. وأرقام الاستخدام إحنا قلنا صراحة إنها لسه قبل الإطلاق.

إيه اللي لسه مفتوح؟

حاجة واحدة أساسية: التطبيق لسه بيتّصل بقاعدة البيانات بحساب واسع الصلاحيات، والمفروض ينزل ل role محدود. إحنا عارفينها وموثّقينها، وهي مكتوبة على شريحة ال remediation نفسها.

هل ال Clinic Assistant بيشفو السجّلات الطبية؟

لأ. ال reason بتاع الزيارة — يعني وصف المشكلة — مش بيترجع للاستقبال أصلاً من ال API. بيشفو مين جاي، إمتى، بأنهي حيوان، ورقم تليفون. وكمان لو الطبيب عطل الحساب، بيتقفل في نفس اللحظة مش لما ال token يخلص.

إيه أصعب مشكلة تقنية قابلكم؟

ال concurrency. حاجتين: حجز آخر مقعد في class من شخصين في نفس اللحظة — اتحلّت ب row lock صريح. وميزانية ال AI المشتركة بين نسخ serverless مش شايقة بعضها — اتحلّت ب semaphore في PostgreSQL نفسه. واللاتنين اتأكّدوا باختبارات concurrency حقيقية على قاعدة بيانات فعلية، مش بالتفكير النظري.